EECE 7205: Fundamentals of Computer Engineering Multi-Course and Exam Scheduler - Iteration #02

Nashita Hasan, Katherine Tse
Email: hasan.nas@northeastern.edu, tse.kat@northeastern.edu
NUID: 002750189, 003936627
Due: October 29, 2026

1 Problem Statement and Proposed Solution

1.1 Problem Statement

- **Problem**: Courses and exams need to be scheduled so that there are not two in the same room and there are not students or professors with overlapping events.
- **Users**: The University Administration Team would use it for general course planning and professors would use it for exam scheduling.
- **Importance**: There are multiple cases and reasons why this problem is important. Scheduling conflicts affect students ability to learn fully. For example, if a student has an exam at the same time as a class, they are forced to either miss the class or the exam which compromises their learning. Professors also cannot teach and hold exams at the same time. They would have to pick between either teaching or being present to answer questions during exams. Additionally, two classes or exams cannot be held in the same space. This would create a suboptimal testing or classroom environment that would affect students' ability to learn and a professor's ability to teach.
- **Relevance**: One single change can create a rippling effect of conflicts within a given schedule which can be time consuming if handled manually. Having an automated process, implemented in C++, will assist the administration and instructor staff in solving scheduling conflicts promptly.

1.2 Proposed Solution

- **Main Features**:
  - Professor overlap checking (i.e. one professor cannot teach/hold an exam at the same time)
  - Student overlap checking (i.e. one student cannot attend two classes/exams at the same time)
  - Room scheduling conflict checking (i.e. one room cannot hold two classes/exams at the same time)
- **Expected Inputs**: A datafile containing Information on exams and courses that include the professor and student list; the number of available rooms and the time slot range for scheduling.
- **Expected Outputs**: All courses and exams with their assigned room and time displayed neatly.

1.3 Why Selected

As students, this program relates directly to things that we experience. Additionally, we are interested in gaining exposure to and improving our skills in new algorithms and techniques such as graph coloring, greedy algorithms, and backtracking.

2 Team Structure, Contributions, and Objectives

2.1 Team Objectives

- Develop an application that successfully reads a file, performs some analysis, and produces a readable result
- Successfully apply concepts from class to solve a practical issue
- Utilize the graph data structure
- Analyze and understand graph coloring, greedy algorithms, and backtracking in order to select the best option to implement
- Test the program on a larger data set
- Improve our GitHub and collaborative working skills

2.2 Team Member Contribution This document was written together in an in-person meeting.

- **Nashita Hasan**:
  - Completed: Discussed problem and solution overview, edited sections of the document alongside team member, brainstormed solution/application details, broke down workflow in pseudocode, wrote individual learning
  - Planned: Research graph coloring, backtracking, and greedy algorithms
- **Katherine Tse**:
  - Completed: Brainstormed and discussed all content sections of the document, wrote and edited sections of document alongside team, formatted documentation and pseudocode, set up GitHub repository, wrote individual learning objects and experience
  - Planned: Research graph coloring, backtracking, and greedy algorithms

2.3 Individual Learning Objectives

- **Nashita Hasan**:
  - Gain more experience with object-oriented programming
  - Design programs using functions and parameter passing
  - Learn how to process data files in C++ and analyze the input to return a desired output

- Develop an understanding of new algorithms/techniques such as graph coloring, greedy algorithms, and backtracking
- Gain more exposure to GitHub and how to properly utilize the tool to optimize programming in a team
- Learn how to efficiently debug code through running test cases

- **Katherine Tse**:

- Read and parse input files (text or csv)
- Implement proper error catching through project (ex. checking if the file opens properly)
- Understand graphs, graph coloring, backtracking, and greedy algorithms and implement the best graph strategy for the problem
- Properly format output based on given input
- Debug program through understanding error messages and utilizing a variety of test cases
- Utilize Git branches, commits, and pull requests and navigate merge conflicts and other Git related errors

3 Programming Design and GitHub Collaboration

3.1 Initial Program Design

- **Main Features**:

- Professor overlap checking
- Student overlap checking
- Room scheduling conflict checking

- **Proposed Data Structures**:

- *Graphs*: Graphs allow us to connect nodes using relevant information such as overlapping professors and students. This will allow us to easily analyze potential constraints between two classes/exams.
- *Vectors*: A vector allows us to store lists of items in a neat and organized way. We will use this to store information as it is read from the datafile. Additionally, this will allow us to keep track of all students that are taking a class/exam.

- **Main Functions**:

- loadData(): Load and read course/exam information from the datafile
- buildGraph(): Load a graph data structure with the information read from the datafile
- Class - course/exam: A class to store all information associated with a course/exam; is a node in the graph

- Class - time: A class to store time slots represented as a start value and end value
- Class - graph: A class to store the graph data structure that connects all the nodes (a vector of nodes)
- checkStudentOverlap(): Returns true if two courses/exams have a student in both
- checkProfessorOverlap(): Returns true if a professor would be teaching two courses/exams at the same time
- assignTime(): Analyze the graph and utilize the proper algorithm to assign each course/exam a time slot
- assignRoom(): Analyze the graph and utilize the proper algorithm to assign each course/exam a room
- displayResults(): Display the resulting schedule with no conflicts

- **Main Program Flow**

Algorithm 3.1 Main Program Flow

```
1: function MAIN
2:     nodes[ ] ← LoadFile(file_name)
3:     graph ← buildGraph(data)
4:     assignTime(graph)
5:     assignRoom(graph)
6:     displayResults(graph)
7: end function
```

- **Build Graph Pseudocode**

Algorithm 3.2 Build Graph (inside graph class)

```
1: function buildGraph(nodes)
2:     n ← nodes.SIZE()
3:     for i ← 0 to n − 1 do
4:         curr ← nodes[i]
5:         graph.APPEND(curr)
6:         for j ← 0 to graph.size() − 2 do
7:             compareTo ← graph[j]
8:             if checkProfessor(curr, compareTo) = TRUE then
9:                 curr.edges.APPEND(compareTo)
10:                compareTo.edges.APPEND(curr)
11:                continue
12:            end if
13:            if checkStudent(curr, compareTo) = TRUE then
14:                curr.edges.APPEND(compareTo)
15:                compareTo.edges.APPEND(curr)
16:            end if
17:        end for
18:    end for
19:    return graph
20: end function
```

- **Testing Plan**
  - *Test 1* Small Data Set with 3 courses/exams where two courses/exams having overlapping student/professor
    - Expected Result: A schedule where the two overlapping courses/exams are not at the same time and any courses/exams at the same time are not in the same room.
  - *Test 2* Small Data Set with 3 courses/exams where no courses/exams having overlapping student/professor
    - Expected Result: A schedule where any courses/exams at the same time are not in the same room. (courses/exams could be at same time)
  - *Test 3* Small Data Set with 3 courses/exams where all courses/exams having overlapping student/professor
    - Expected Result: A schedule where all courses/exams are at different times (courses/exams could all be in the same room)
  - *Test 4* Large Data Set for Comprehensive Test (Over 20)
    - Expected Result: A schedule where the two overlapping courses/exams are not at the same time and any courses/exams at the same time are not in the same room.

3.2 GitHub Setup

- **GitHub Link**: https://github.com/Kat24-4/EECE7205FinalProject_HasanTse
- **GitHub Repository Name**: "EECE7205FinalProject_HasanTse"
- **Repository Owner**: Katherine Tse
- **Branch Strategy**: Each member will work on their own personal branch and submit a pull request for the other team member to review.
- **Commit Messages**: Each member is expected to commit at least once per iteration unless coding together as a pair, which in that case will be stated in a clear message.
- **Pull Request and Code Review Process**: Completed work will be submitted as a pull request from a personal branch into main with a brief description of what was worked on. The other member will review and approve this code in a timely manner before merging.
- **Repository Responsibilities**: Both team members will keep their branches updated, regularly commit, and review pull requests promptly. Since there are only two team members, overall repository responsibilities will be share (one member is not in charge of the GitHub).
- **README Contents**: The README will include the project title, names of each team member, and sections for the following content: project description, progress/iteration updates, build instructions, run instructions, and example execution. The README will be update after every iteration.

4 Tools and Programming Language

Note: Any future tool/language changes will be added to documentation promptly

4.1 Programming Language

- C++ (Primary language for application)

4.2 Development Tools

- **Compiler**: GCC/g++
- **IDE**: Visual Studio Code (VSCode)
- **Version Control and Collaboration**: Git and GitHub
- **Debugging Tools**: VSCode built-in C++ debugger; Claude (for error code message deciphering and explanations)

4.3 Data and Testing

- **Input**: Data or CSV file
- **Output**: Data/CSV file or command line printing
- **Small Sample Datasets**: personally generated/fabricated (3 items each)
- **Large Sample Datasets**: personally generated or sourced from Professor

4.4 Team Experience

- **Nashita Hasan**: This project will help me develop my C++ programming and my design/problem-solving skills, as well as introduce GitHub collaboration proper debugging.
- **Katherine Tse**: This project will strengthen my existing C++ and collaboration skills across all of the above tools and methods and introduce AI as a development tool in project work.